Universidad Autónoma de Baja California

Facultad de Ingeniería, Mexicali

Ingeniería En Computación

Carlos Alberto Aguilar Avelar

Microcontroladores

Práctica 5 Medidor de Distancia Ultrasonico (Timers)

Alejandro Castro Rentería 1183297

Juan Carlos Alvarado Rucobo 1180084

Baja California, Mexicali 10/04/2024

Introducción.

En esta práctica realizaremos el funcionamiento del sensor ultrasónico teniendo en cuenta que usaremos el sensor que gracias al creador las señales emitidas son lineales

¿Qué es una pantalla LCD?

Una pantalla de cristal líquido o LCD (sigla del inglés liquid-crystal display) es una pantalla plana y delgada formada por un número de píxeles en color o monocromos colocados delante de una fuente de luz o reflector. A menudo se utiliza en dispositivos electrónicos de baterías, ya que utiliza cantidades muy pequeñas de energía eléctrica.

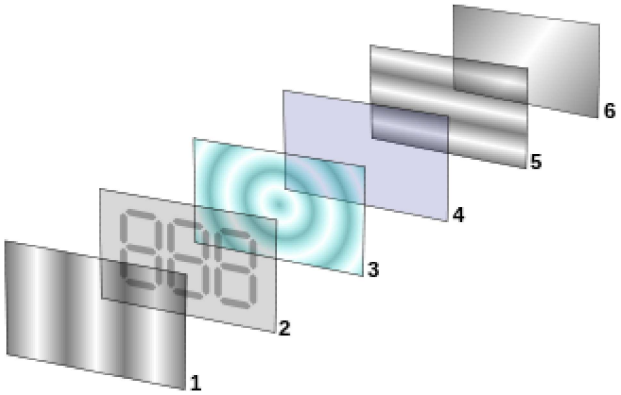

Figura 1- Imagen mostrando las capas de una pantalla LCD.

¿Qué es un sensor ultrasónico?

Un sensor óptico tiene un transmisor y receptor, mientras que un sensor ultrasónico utiliza un elemento ultrasónico único, tanto para la emisión como la recepción. En un sensor ultrasónico de modelo reflectivo, un solo oscilador emite y recibe las ondas ultrasónicas, alternativamente. Esto permite la miniaturización del cabezal del sensor.

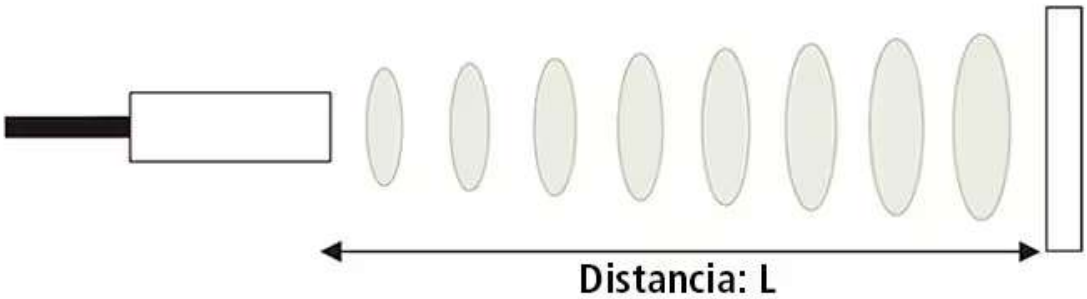

Figura 2- Imagen mostrando el funcionamiento del sensor y recibiendo la senal de regreso.

Timer de Captura.

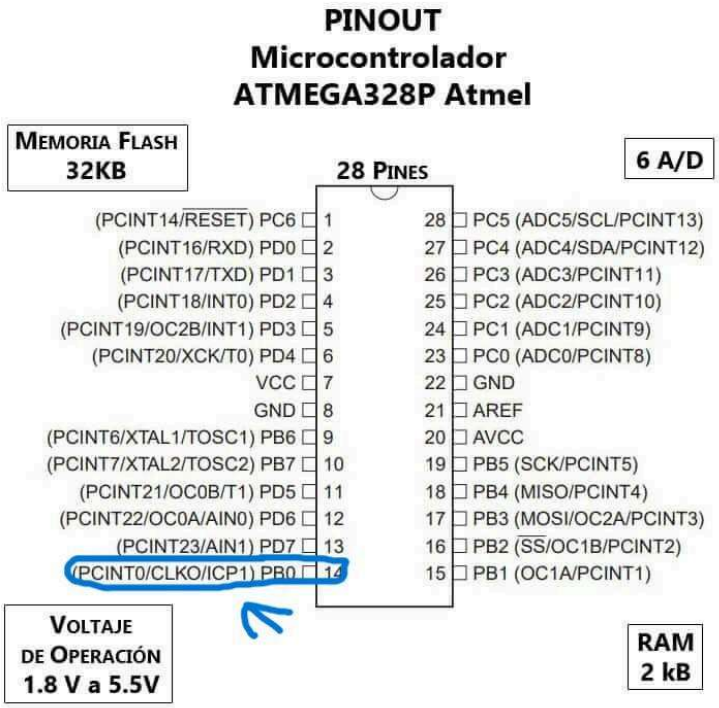

Figura 3- Diagrama del pinout del atmega328p donde se especifica el pin ICP1 (pin puerto de captura).

Los timers o temporizadores son características estándar de casi todos los microcontroladores, así que es muy importante aprender su uso. Los microcontroladores AVR tienen temporizadores muy poderosos y multifuncionales, pueden medir tiempo, generar frecuencias, contar eventos externos, hacer comparaciones, sincronizar tareas, generar una onda PWM, etc.

El microcontrolador ATmega328p dispone de tres timers. Mientras el timer1 es de 16 bits, el timer0 y el timer2 son de 8 bits. Dichos timers son la base por la que el dispositivo de este proyecto puede realizar tareas de forma síncrona con precisiones de centenas de microsegundos y de forma simultánea. En este proyecto, los timers permiten realizar de forma “simultánea” el parpadeo del LED rgb, el muestreo de la señal del ECG y la medición del estado de la batería.

Un timer se trata de un contador cuya entrada está conectada al reloj del microcontrolador. Por este motivo, el timer aumenta su valor de forma regular y, de este modo, permite medir el tiempo con una gran precisión y ejecutar tareas en espacios de tiempos regulares. Dado que el principio de funcionamiento es muy simple, se explicará en este apartado la configuración para el timer1, el timer de 16 bits del microcontrolador, siendo de forma muy semejante la configuración de los timers restantes.

Configuración del Timer1

En primer lugar, es necesario desactivar las interrupciones, para evitar que tenga lugar una interrupción hasta que el timer esté completamente inicializado. A continuación, se resetean los registros de control del timer, para después realizar un set a los bits de configuración que se necesitan, como la selección del preescalador y la activación del modo CTC. En el modo CTC, clear timer on compare match, se realiza un reseteo del contador del timer una vez que el valor del contador (registro TCNT1) alcanza el valor del registro OCR1A. En este momento tiene lugar la llamada de la función callback de la interrupción y el contador comienza desde cero [26].

/* desactivar las interrupciones*/ cli();

/* set los registros TCCR1A y TCCR1B al valor 0 */ TCCR1A = 0;

TCCR1B = 0;

/* inicializar el valor del Contador del timer a 0 */ TCNT1 = 0;

/* set el registro compare match para incrementos de 5 ms value = (16*10^6) / (1*1024) - 1 (must be <65536) */ OCR1A = 10000; /* activar el modo CTC */ TCCR1B |= (1 << WGM12); /* 256 prescaler */ TCCR1B |= (1 << CS11);

/* enable timer compare interrupt */ TIMSK1 |= (1 << OCIE1A);

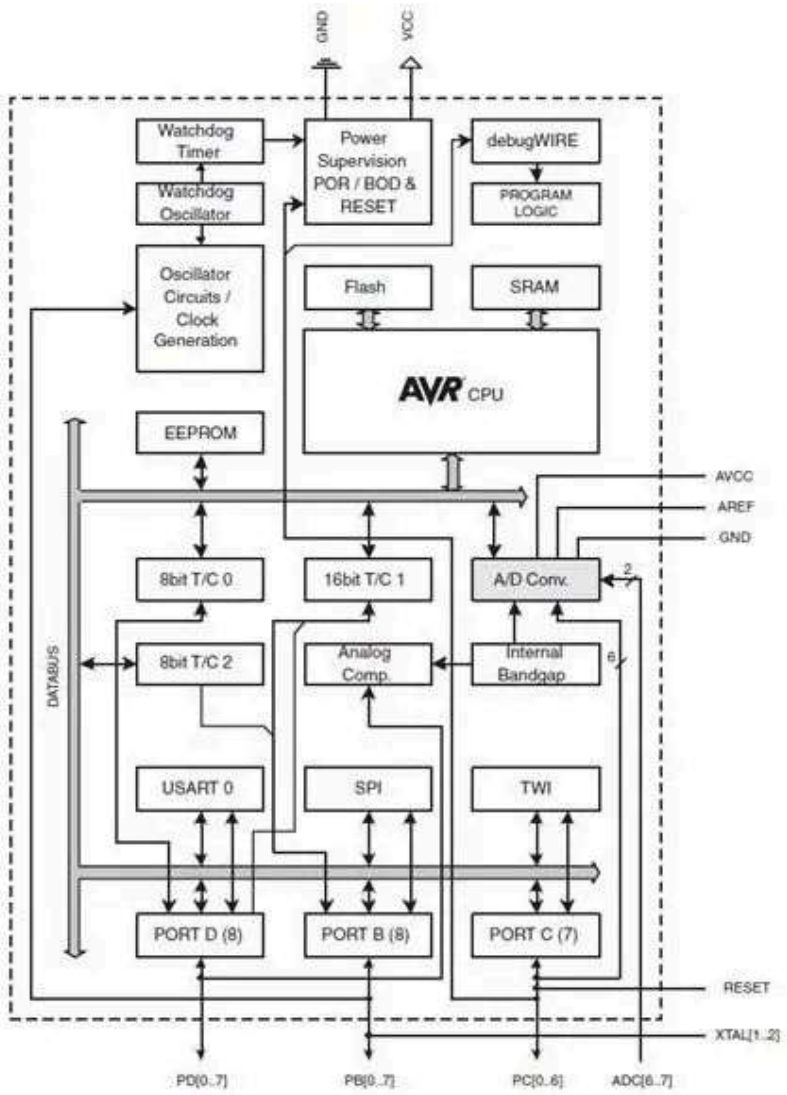

Figura 3- Diagrama representando las conexiones internas del microcontrolador atmega328p.

/* allow interrupts */ sei();

Para la correcta configuración del timer será necesario conocer el valor que debe tener el registro OCR1A y el preescalador que se debe de usar para conseguir que la interrupción se dispare en el momento deseado. Es fácil deducir la siguiente expresión:

Frecuencia de interrupción (*Hz*) = *CLK ATmega*328*p* (16 *MHz*)

preescalador · (*registro Compare match* + 1)

Y despejando de la expresión anterior:

registro Compare match (*OCR*1*A*) = 16.000.000

$preescalador \cdot frecuencia\ deseada(Hz)-$

Desarrollo

Elabore un sistema embebido para que opere como un medidor de distancia ultrasónico digital.

<u>El dispositivo debe contar con las siguientes características:</u>
1. Una pantalla LCD para mostrar la distancia medida.
2. Un botón/interruptor que cambie las unidades entre cm o metros.
3. Un botón/interruptor que active/desactive el modo sensor de colisión.
4. Una alarma auditiva para el modo sensor de colisión.

<u>El dispositivo debe operar de la siguiente manera:</u>
• En la pantalla siempre se muestra la medición de distancia, que se actualiza 5 veces por segundo.
• El rango de detección debe ser de 2 a 100 cm.
• En modo de sensor de colisión, la segunda fila de la pantalla LCD se llena de manera proporcional a la distancia y una alarma auditiva (buzzer) indica cuando un objeto se acerca a menos de 10 cm.
• No debe presentar bugs o comportamientos inesperados.

Primero se explica qué pines y mostramos que librerías se usan:

Antes que nada se configura la Pantalla LCD, para facilitar esto se añade la librería compartida del profe, el mismo que se usó para la práctica anterior (LCD.h y LCD.c).
Pues esta librería permite configurar la pantalla y de manera fácil imprimir en ella. Lo único malo es que usa todo el PORTC, por lo que debe usar los otros pines para los demás funcionamientos.

Cómo se usará captura, por obligación se asigna el pin B0 como la entrada que recibe el echo del sensor HC-SR04, se decidió usar B1 para la salida al trigger, esto para que este a un lado del echo.

Para las interrupciones de control de los modos y el buzzer se usó algunos pines del PORTD:

```C/C++
#define F_CPU 16000000
#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <util/delay.h>
#include "LCD.h"
#include "LCD.c"
#define LOW 0
#define HIGH 1
//PORTB PINS
#define TRIGGER_PIN 1
#define ECHO_PIN 0
//PORTD PINS
#define BUZZER_PIN 2
```

```
#define COLISION_BUTTON_PIN 4
#define MODE_BUTTON_PIN 3
```

Ahora sigue configurar la dirección de datos de los pines, el timer1 en modo captura y interrupciones que se usan, para la asignación de pines se a cabo en una función y lo demás en el main:

```C/C++
void preparePins() {
        DDRB |= (1 << TRIGGER_PIN); // Configurar el pin TRIGGER como salida
        DDRB &= ~(1 << ECHO_PIN); // Configurar el pin ECHO como entrada
        PORTB |= (0 << ECHO_PIN); //habilitar resistencia pull-down interna en el pin ECHO
        DDRD &= ~(1 << COLISION_BUTTON_PIN); // Configurar el pin del botón como entrada
        PORTD |= (1 << COLISION_BUTTON_PIN); // Habilitar pull-up en el pin del botón
        DDRD |= (1 << BUZZER_PIN); // Configurar el pin del buzzer como salida
        PORTD &= ~(1 << BUZZER_PIN); // Asegurarse de que el buzzer esté inicialmente desactivado
        DDRD &= ~(1 << MODE_BUTTON_PIN); // Configurar el pin MODE_BUTTON como entrada
        PORTD |= (1 << MODE_BUTTON_PIN); //  Habilitar pull-up en el pin del botón
}
```

En Main:

```C/C++
lcd_init();
        preparePins();
        //configuracion timer
        TCCR1B |= (1 << ICNC1) | (0 << ICES1) | (1 << CS12) | (0 << CS11) | (1 << CS10); // Configurar el
Timer1 para captura de entrada con ruido cancelado, flanco de subida, y preescalador 1024
        TIFR1 = 1 << ICF1; // Limpiar bandera de interrupción de captura de entrada
        TIMSK1 |= (1 << ICIE1); // Habilitar interrupción de captura de entrada
        //configuracion interrupciones
        PCICR |= (1<<PCIE2); //habilitar interrupciones en PORTD
        PCMSK2 |= (1<<PCINT19)|(1<<PCINT20); //activa D3 y D4 para interrupcion
        sei();

void preparePins() {
        DDRB |= (1 << TRIGGER_PIN); // Configurar el pin TRIGGER como salida
        DDRB &= ~(1 << ECHO_PIN); // Configurar el pin ECHO como entrada
        PORTB |= (0 << ECHO_PIN); //habilitar resistencia pull-down interna en el pin ECHO
        DDRD &= ~(1 << COLISION_BUTTON_PIN); // Configurar el pin del botón como entrada
        PORTD |= (1 << COLISION_BUTTON_PIN); // Habilitar pull-up en el pin del botón
        DDRD |= (1 << BUZZER_PIN); // Configurar el pin del buzzer como salida
        PORTD &= ~(1 << BUZZER_PIN); // Asegurarse de que el buzzer esté inicialmente desactivado
        DDRD &= ~(1 << MODE_BUTTON_PIN); // Configurar el pin MODE_BUTTON como entrada
        PORTD |= (1 << MODE_BUTTON_PIN); //  Habilitar pull-up en el pin del botón
}
```

Ahora se demuestra lo más importante, cómo se lleva a cabo la lectura del sensor ultrasónico y la conversión a distancia.

Para leer el sensor, primero se manda un pulso pequeño al trigger, esto hace que haga una lectura el sensor y regrese cuanto duro la onda echo del sensor mandando un pulso positivo proporcional a la duración.

Existen algunas maneras diferentes para llevar a cabo esto:

- La primera forma involucra usar una fórmula de velocidad despejada para regresar la distancia, consiste en calcular la duración del echo en microsegundos, vamos calculando cuánto tardo en recibir el echo lo que es la señal del trigger esperando cierto tiempo usamos la demostración que está en la presentación de la presente práctica que es el tiempo en el que esperamos los 10 microsegundos los 8 ciclos de 40 Khz y luego vemos el flanco de bajada.
- Debido al comportamiento del sensor se puede sacar una función de tendencia, esto se hace por medio de comparar los microsegundos que duro el echo con la distancia en que se encuentra el objeto detectado, se llena una tabla tomando algunas muestras, con ayuda de excel se puede graficar y sacar la formula de tendencia de forma automática, se usa esta formula en el código asegurando que este despejado correctamente.

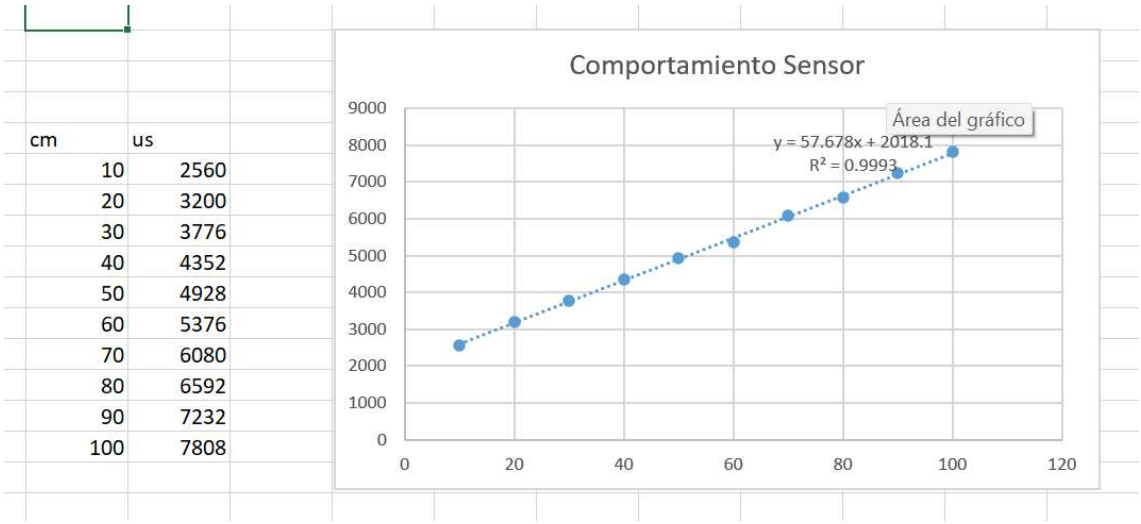

Figura 4- gráfica creada a partir de los valores la tabla que representa la relación duración de echo en microsegundos con distancia en cm, se usa excel para hacer más fácil sacar la formula de tendencia.

En la forma simulada se llevó a cabo el primera forma, se hace por pasos todo: primero aseguramos que no se detecte un pulso positivo en echo, si no es el caso se activa el trigger ,se espera que termine el ruido, se pone en 0 el timer1, ahora debe esperar que se lleve a cabo el flanco de bajada que indica la terminación del echo, al momento de activarse la interrupción de captura, se cambia la bandera para entrar a lo que sigue, leer el valor guardado en el registro de captura (este indica la duración del echo en ciclos) se saca el calculo de la duración en microsegundos y se calcula la distancia usando la fórmula que viene en el datasheet (us/58 = distancia(cm)).

```
C/C++
sei();

        while (1) {
```

```
if (!(PINB & (1 << ECHO_PIN)) && (op_flag == 0)) { // Verificar si el pin de eco está en bajo
        op_flag = 1;
        PORTB |= (1 << TRIGGER_PIN); // Establecer trigger en alto
        _delay_us(10); // Esperar 10 microsegundos
        PORTB &= ~(1 << TRIGGER_PIN); // Establecer trigger en bajo
        _delay_us(200);
        TCNT1 = 0;

        } else if (op_flag == 2) {
        op_flag = 0;
        cvalue = ICR1;
        char buffer[16];
        lcd_clear();

        tvalue = (cvalue * 1000000.0) / 15625.0;
        distance = tvalue / 58; //(proteus)
        //distance = (tvalue-2018.1)/57.678; //(fisico) Ajustar la fórmula según la velocidad
del sonido


        if (flag_CM_o_MT==false)
        {
                dtostrf(distance, 0, 2, floatString);
                snprintf(buffer, sizeof(buffer),"Dist: %s cm",floatString);
        }
        else
        {
                distance_meters = distance / 100.0; // Convertir a metros
                dtostrf(distance_meters, 0, 2, floatString);
                snprintf(buffer, sizeof(buffer), "Dist: %s m", floatString);
        }

        lcd_goto_xy(0, 0);
        lcd_write_word(buffer);

        if(flag_Buzzer==true)
        {
                if (distance <= 10)
                {
                        warning_i=distance*10;
                        PORTD |= (1<<BUZZER_PIN);
                        for(int i=0; i<warning_i; i++)
                        {
                                _delay_ms(1);
                        }
                        PORTD &= ~(1 << BUZZER_PIN);
                }
                char segment_string[16]="                ";
                if(distance>100)segmentos=16;
                else segmentos = distance/6.25;
                for(int i=0;i<segmentos;i++)segment_string[i]='0' ;
                lcd_goto_xy(1,0);
                lcd_write_word(segment_string);
```

```
                        }
                        else
                        {
                                PORTD &= ~(1 << BUZZER_PIN);
                        }
                        _delay_ms(200);
                }
        }
}
```

Dentro del ciclo se estará tomando las distancias usando lo que es el timer de captura usando las interrupciones para checar que hubo una interrupción use las condiciones dentro del while para ir checando si presionaron el botón use ciclos para esperar si el usuario oprime el botón en cuyo caso no sea así entonces continuará usando distancias el timer de captura lo tomamos primero mandando la señal del trigger al echo como nuestro sensor ultrasónico recibe de cierto ángulo de 15 grados las señales al rebotar con una pared entonces obteniendo estos datos calculamos la distancia dividiendo entre 58 para obtener la distancia en centímetros y simplemente en la parte de interrupción para mostrar según la distancia pues se estará mostrando los cuadros que yo use ceros para mostrar cómo qué tanta distancia hay entre el sensor y la pared y sonará un bip si se usa el botón de colisión si está a 10 cm o menos qué será intermitente.

Siendo realista para la parte del buzzer tenía la idea de crear la librería tone como en arduino pero no sabia mas que usar un ciclo que vaya cambiando los estados del buzzer usando una variable como si fuera tiempo pero mejor se optó por meter un delay en un ciclo por segundo.

Práctica Simulada en Proteus.

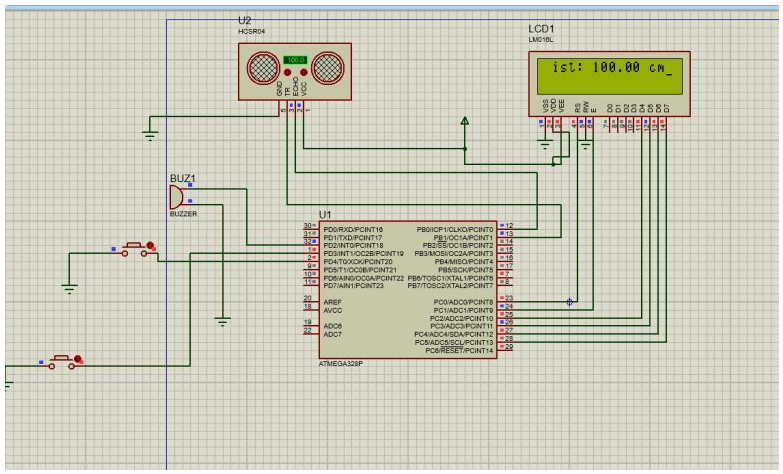

Figura 5- Simulación del circuito compilando el código de Microchip Studio cómo funciona el sensor en la medida de 100cm y veremos el modo de colisión.

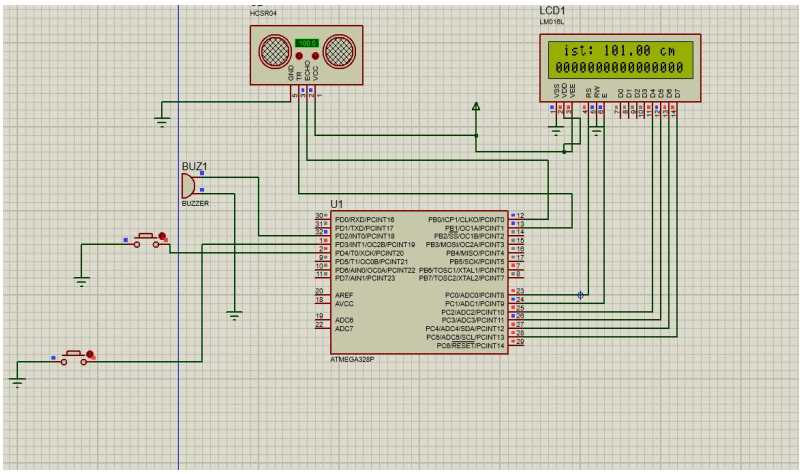

Figura 6- Aquí vemos el modo colisión usando los 100 cm como referencia y muestra que tan lejos está con los ceros.

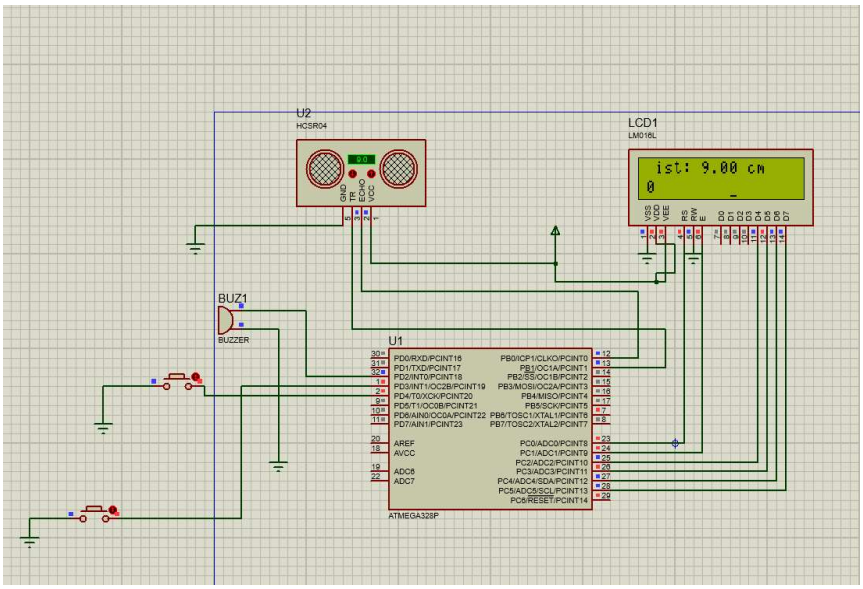

Figura 7- En esta parte vemos como el modo colisión funciona de 10 o menor cm y se prende y apaga el buzzer y cambia la forma de mostrarlo en el LCD.

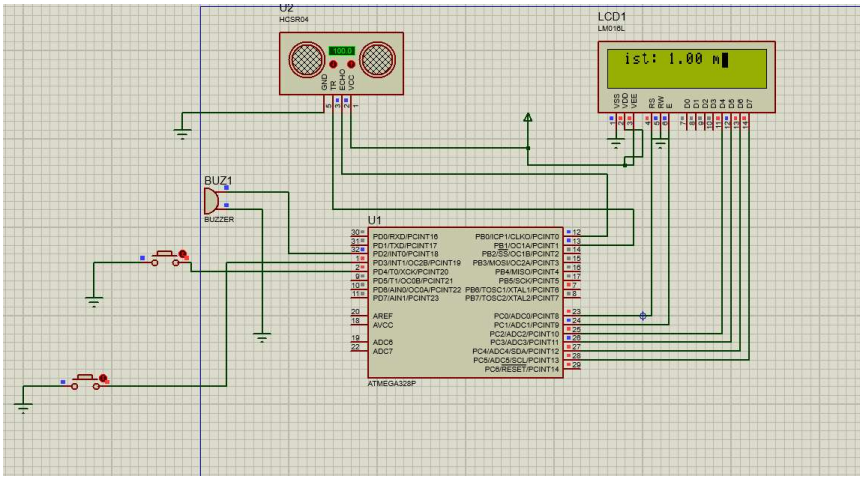

Figura 8- En esta imagen vemos como cambia de cm a metros usando los 100 cm de referencia.

Funcionamiento del Circuito Físico.

Figura 9- En esta imagen vemos el circuito armado y en funcionamiento usando el arduino como programador y mandando los resultados al lcd.

Videos:

https://drive.google.com/file/d/1DWSxlB40NVznRqPDNniTFruhqj7TVqeY/view?usp=sharing
10000000_7511294128949212_7578851488650781318_n.mp4
https://drive.google.com/file/d/1n4wmMsBNGR6ScdajZKFxZenpctuyC5WE/view?usp=sharing

Conclusiones.

En conclusión podemos determinar que esta práctica salio como lo esperado ya que podemos gracias a el uso de timers capturar la distancia del sensor usando el contador interno y usando el escalador que en este caso teníamos una diferencia muy grande ya que usamos un escalador de 1024 ya que si usamos uno mas pequeño podriamos tener mas exacta la medición como por ejemplo usar 128 o 64, realmente al declarar el timer nosotros vamos checando el flanco de bajada al final para calcular la distancia con respecto a ese tiempo para mostrar estos valores usamos una librería para mostrar un string en forma de flotante y decirle que cantidad de decimales queremos mostrar.

Hay que tomar en consideración que fuente usamos para cargar el microcontrolador, un cargador de celular tiene mucho ruido, esto puede provocar comportamientos negativos en los componentes negativos, es recomendado usar una batería o la misma conexión del arduino con la computadora para alimentar el circuito.

Referencias.

De Alfreedom, V. T. L. E. (2020, 16 abril). *AVR programación en C – 10 Timer/Counter0 del ATmega16/32 – Parte 2 (Modo CTC)*. Vida Embebida. https://vidaembebida.wordpress.com/2014/10/13/avr-programacion-en-c-09-timercounter0-del-atmega16-parte-2/

De Alfreedom, V. T. L. E. (2018, 15 abril). *AVR programación en C – 06 Interrupciones en los AVR*. Vida Embebida. https://vidaembebida.wordpress.com/2014/07/29/avr-programacion-en-c-06-interrupciones-en-los-avr/

Gandhi, M., Gandhi, M., & Gandhi, M. (2019, 19 diciembre). *Qué es un sensor ultrasónico y para qué sirve*. AUTYCOM - AUTYCOM. https://www.autycom.com/que-es-un-sensor-ultrasonico-y-para-que-sirve/

¿Qué es un sensor ultrasónico? | *Fundamentos del sensor: Guía de sensores para fábricas clasificados por principios* | *KEYENCE*. (s. f.). https://www.keyence.com.mx/ss/products/sensor/sensorbasics/ultrasonic/info/

Anexos

codigo completo:

```
C/C++

#define F_CPU 16000000
#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <util/delay.h>
#include "LCD.h"
#include "LCD.c"
#define LOW 0
#define HIGH 1
//PORTB PINS
#define TRIGGER_PIN 1
#define ECHO_PIN 0
//PORTD PINS
#define BUZZER_PIN 2
#define COLISION_BUTTON_PIN 4
#define MODE_BUTTON_PIN 3

uint8_t op_flag = 0;
bool flag_Buzzer = false;
bool flag_CM_o_MT = false;
uint16_t tvalue;
float distance;
uint16_t cvalue;
char floatString[8];
int warning_i;
int segmentos;
float distance_meters;

void preparePins(void);

ISR(TIMER1_CAPT_vect) {
	if (op_flag == 1) {
		op_flag = 2; // Marca que se ha realizado una captura exitosa
	}
}

ISR(PCINT2_vect)
{
	if(!(PIND & (1 << COLISION_BUTTON_PIN)))
	{
		_delay_ms(50);
```

```
		flag_Buzzer=!flag_Buzzer;
		while (!(PIND & (1 << COLISION_BUTTON_PIN)));
	}
	if(!(PIND & (1 << MODE_BUTTON_PIN)))
	{
		_delay_ms(50);
		flag_CM_o_MT=!flag_CM_o_MT;
		while (!(PIND & (1 << MODE_BUTTON_PIN))); // Esperar a que el
botón sea liberado
	}
}

int main(void) {
	lcd_init();
	preparePins();
	//configuracion timer
	TCCR1B |= (1 << ICNC1) | (0 << ICES1) | (1 << CS12) | (0 << CS11) | (1
<< CS10); // Configurar el Timer1 para captura de entrada con ruido
cancelado, flanco de subida, y preescalador 1024
	TIFR1 = 1 << ICF1; // Limpiar bandera de interrupción de captura de
entrada
	TIMSK1 |= (1 << ICIE1); // Habilitar interrupción de captura de
entrada
	//configuracion interrupciones
	PCICR |= (1<<PCIE2); //habilitar interrupciones en PORTD
	PCMSK2 |= (1<<PCINT19)|(1<<PCINT20); //activa D3 y D4 para
interrupcion
	sei();

	while (1) {


		if (!(PINB & (1 << ECHO_PIN)) && (op_flag == 0)) { // Verificar
si el pin de eco está en bajo
			op_flag = 1;
			PORTB |= (1 << TRIGGER_PIN); // Establecer trigger en
alto
			_delay_us(10); // Esperar 10 microsegundos
			PORTB &= ~(1 << TRIGGER_PIN); // Establecer trigger en
bajo
			_delay_us(200);
			TCNT1 = 0;

			} else if (op_flag == 2) {
			op_flag = 0;
			cvalue = ICR1;
			char buffer[16];
			lcd_clear();
```

```
                    tvalue = (cvalue * 1000000.0) / 15625.0;
                    //distance = tvalue / 58; //(proteus)
                    distance = (tvalue-2018.1)/57.678; //(fisico) Ajustar la
fórmula según la velocidad del sonido


                    if (flag_CM_o_MT==false)
                    {
                           dtostrf(distance, 0, 2, floatString);
                           snprintf(buffer, sizeof(buffer),"Dist: %s
cm",floatString);
                    }
                    else
                    {
                           distance_meters = distance / 100.0; // Convertir a
metros
                           dtostrf(distance_meters, 0, 2, floatString);
                           snprintf(buffer, sizeof(buffer), "Dist: %s m",
floatString);
                    }

                    lcd_goto_xy(0, 0);
                    lcd_write_word(buffer);

                    if(flag_Buzzer==true)
                    {
                           if (distance <= 10)
                           {
                                  warning_i=distance*10;
                                  PORTD |= (1<<BUZZER_PIN);
                                  for(int i=0; i<warning_i; i++)
                                  {
                                         _delay_ms(1);
                                  }
                                  PORTD &= ~(1 << BUZZER_PIN);
                           }
                           char segment_string[16]="                ";
                           if(distance>100)segmentos=16;
                           else segmentos = distance/6.25;
                           for(int i=0;i<segmentos;i++)segment_string[i]='0'
;
                           lcd_goto_xy(1,0);
                           lcd_write_word(segment_string);

                    }
                    else
                    {
```

```
                        PORTD &= ~(1 << BUZZER_PIN);
                    }
                    _delay_ms(200);
            }
        }
}

void preparePins() {
        DDRB |= (1 << TRIGGER_PIN); // Configurar el pin TRIGGER como salida
        DDRB &= ~(1 << ECHO_PIN); // Configurar el pin ECHO como entrada
        PORTB |= (0 << ECHO_PIN); //habilitar resistencia pull-down interna en
el pin ECHO
        DDRD &= ~(1 << COLISION_BUTTON_PIN); // Configurar el pin del botón
como entrada
        PORTD |= (1 << COLISION_BUTTON_PIN); // Habilitar pull-up en el pin
del botón
        DDRD |= (1 << BUZZER_PIN); // Configurar el pin del buzzer como salida
        PORTD &= ~(1 << BUZZER_PIN); // Asegurarse de que el buzzer esté
inicialmente desactivado
        DDRD &= ~(1 << MODE_BUTTON_PIN); // Configurar el pin MODE_BUTTON como
entrada
        PORTD |= (1 << MODE_BUTTON_PIN); //  Habilitar pull-up en el pin del
botón
}
```